

Prediction Arena (Kalshi) — File Layout Plan

Context

We're scaffolding a Kalshi-only prediction-market trading benchmark inspired by Arcada Labs' *Prediction Arena* (arXiv:2604.07355). It runs Claude vs Codex head-to-head as independent agents on 19 curated Kalshi markets, with \$250 per agent and a global paper/execute mode toggle. Approach is outcome-driven (every trade requires a probability-of-outcome thesis) but the agents can use flow / volume / order book to refine the thesis. Hold-to-settlement default with a volume-spike defensive exit.

This plan covers the **file/folder layout** only. The verbatim system prompt is the next deliverable, not part of this plan.

Target location

`toolshed/users/llewis/prediction_arena/` (inside llewis user folder — blanket permission per CLAUDE.md).

Pre-implementation cleanup

A partial scaffold landed on disk before plan mode flipped on. **User has confirmed: revert all and re-scaffold cleanly.** First implementation action is to delete

`toolshed/users/llewis/prediction_arena/` entirely, then rebuild per this plan.

Files/dirs to delete on cleanup:

- `prediction_arena/README.md`
- `prediction_arena/.env.example`
- `prediction_arena/.gitignore`
- `prediction_arena/requirements.txt`
- `prediction_arena/config/__init__.py`
- All empty subdirectories created under `prediction_arena/`

Full target tree

<pre> prediction_arena/ ├── README.md ├── requirements.txt ├── ├── dotenv, pytest ├── .env.example ├── Supabase ├── .gitignore ├── ├── config/ │ ├── __init__.py │ └── settings.py </pre>	Project overview, scope, layout, setup steps anthropic, openai, tavily-python, supabase, psycopg2-binary, httpx, pydantic, python- ARENA_MODE, Kalshi keys, LLM keys, Tavily, .env, __pycache__, logs/*.jsonl, .venv MODE, CAPITAL_PER_AGENT_USD,
---	---

```

CONCENTRATION_PCT,
|
|
|
|
MIN/MAX_HOURS_TO_RESOLUTION,
|
| — markets.py
Financial, no Sports)
| — agents.py
+ codex)
| — cycle.py
|
|
| — prompts/
| — system_prompt.md
|
plan)
| — changelog.md
driving each
|
| — exchange/
| — __init__.py
| — base.py
|
get_volume,
|
| — kalshi_live.py
| — kalshi_paper.py
availability
|
| — factory.py
config.settings.MODE
|
| — llm/
| — __init__.py
| — base.py
tools)
|
| — claude_runner.py
| — codex_runner.py
| — tool_schema.py
providers
|
| — tools/
| — __init__.py
| — trading.py
contracts
| — research.py
result cap
| — notes.py
cap)
| — market_data.py
|

```

MIN_EV_PER_CONTRACT_USD, MIN_EDGE_GAP,
MIN_BOOK_DEPTH_USD,

KELLY_FRACTION_CAP, VOLUME_SPIKE_MULTIPLIER
19 tickers grouped by category (no

AgentConfig dataclass + AGENTS list (claude

MIN/MAX_CYCLE_MINUTES, MAX_TOOL_CALLS,
MAX_CYCLE_SECONDS

Verbatim fundamental-leaning Kalshi prompt
(separate deliverable – not written in this

Track prompt revisions + failure modes

ExchangeAdapter interface:
fetch_markets, get_quote, get_book,

get_price_history, place_buy, get_positions,
get_balance, get_settlements
Hits Kalshi REST/WS API
Simulated fills – honors counterparty

(stricter than Arcada paper's Cohort 2)
Returns adapter based on

AgentRunner interface: run(prompt, ctx,

→ (tool_calls, reasoning, final_message)
Anthropic SDK + tool-use API
OpenAI SDK + tool-use API
Normalize tool-call schemas between

buy(market_id, side, qty) – integer

web_search(query) – Tavily wrapper, no

note_create/search/edit/delete (no per-cycle
cap)

get_order_book, get_recent_volume,
get_price_history, get_market_activity

└─ registry.py	Whitelist enforcement – reject undeclared
tools	
└─ orchestration/	
└─ __init__.py	
└─ scheduler.py	Per-agent timer, uniform random [15, 45]
min, 24/7	
└─ cycle.py	6-step cycle: sync → settle → context →
agent →	
└─ context_builder.py	execute → metrics
+ learning	Assembles market data + portfolio + history
└─ learning_section.py	Deterministic summary of last 30 trades by
category,	
	win/loss patterns
└─ risk/	
└─ __init__.py	
└─ concentration.py	15% cost-basis cap per market vs account
value	
└─ solvency.py	Cash + fees check
└─ ev_filter.py	Reject trades with EV per contract < \$0.02
after fees	
└─ kelly.py	$f^* = (p \cdot b - q) / b$, capped at 0.25 (quarter
Kelly)	
└─ pre_trade_filter.py	Edge gap ≥7%, book depth ≥\$100 both sides,
└─ circuit_breaker.py	hours-to-resolution in [4, 168]
	3x volume spike in 10 min → defensive exit
└─ accounting/	
└─ __init__.py	
└─ mark_to_market.py	$V_t = C_t + \sum(Q_p \cdot B_p^t)$ at bid
└─ netting.py	\$1 per matched pair – (cost basis A + cost
basis B)	
└─ fees.py	Kalshi fee schedule + per-trade fee calc
└─ reconciliation.py	Post-cycle reconcile against exchange state
└─ db/	
└─ __init__.py	
└─ schema.sql	Tables: agents, markets, cycles, trades,
positions,	
└─ client.py	settlements, snapshots, notes, tool_calls
└─ migrations/	Supabase client wrapper
	Versioned schema changes (empty initially)
└─ metrics/	
└─ __init__.py	
└─ snapshot.py	Per-cycle: account value, PnL, win rate, max
DD	
└─ leaderboard.py	Recompute Claude vs Codex ranking after each
cycle	
└─ behavioral.py	Settlement rate, category preference, sizing
dist	

```
|
| └─ scripts/
|   └─ run_arena.py           Main entry – starts scheduler for both
agents
|   └─ run_single_cycle.py    One cycle for one agent (debugging)
|   └─ init_db.py            Apply schema.sql
|   └─ settle_resolved_markets.py  Cron-able settlement processor
|   └─ dump_snapshot.py       Print current state for an agent
|
| └─ tests/
|   └─ __init__.py
|   └─ test_paper_adapter.py   Paper-mode fill logic + counterparty
honoring
|   └─ test_concentration.py   15% cap edge cases
|   └─ test_netting.py        Matched-pair payout math
|   └─ test_mark_to_market.py Bid-priced MTM
|   └─ test_ev_filter.py      $0.02 + 7% edge filters
|   └─ test_kelly.py          Quarter-Kelly sizing
|   └─ test_context_builder.py Context assembly correctness
|   └─ fixtures/              Mock Kalshi responses (JSON)
|
| └─ logs/                    Per-cycle JSONL transcripts (gitignored)
|   └─ (empty until first cycle)
```

Key configuration values (final)

Setting	Value	Source
MODE	paper / execute (env-driven)	User-chosen
CAPITAL_PER_AGENT_USD	250	User-chosen
CONCENTRATION_PCT	0.15	Paper §4.5.1
MIN_EV_PER_CONTRACT_USD	0.02	Added for fee-sensitivity at \$250
MIN_EDGE_GAP	0.07	LunarResearcher thread
MIN_BOOK_DEPTH_USD	100	LunarResearcher (lowered from \$500)
MIN_HOURS_TO_RESOLUTION	4	LunarResearcher
MAX_HOURS_TO_RESOLUTION	168 (7 days)	Widened from LunarResearcher's 48
KELLY_FRACTION_CAP	0.25	LunarResearcher
VOLUME_SPIKE_MULTIPLIER	3.0	LunarResearcher
VOLUME_SPIKE_WINDOW_MIN	10	LunarResearcher

Setting	Value	Source
MIN_CYCLE_MINUTES	15	Paper §3.2
MAX_CYCLE_MINUTES	45	Paper §3.2
MAX_CYCLE_SECONDS	300	Runaway-loop safeguard
MAX_TOOL_CALLS_PER_CYCLE	20	Runaway-loop safeguard

Market universe (19 tickers, 5 categories)

- **Weather (5)**: KXHIGHTNY, KXHIGHTMIA, KXLOWTDEN, KXDSCSNOWM, KXHMONTH
- **Crypto (4)**: KXBTCD, KXETH, KXXRPD, KXSHIBAD
- **Politics (5)**: KXGREENLAND, KXIMPEACH, KXUSAEXPANDTERRITORY, KXTRUMPSAY, KXTRUMPSAYMONTH
- **Entertainment (3)**: KXNETFLIXRANKSHOW, KXOSCARACTO, KXGRAMSOTY
- **Meta/AI (2)**: KXTOPMODEL, KXLAYOFFSYINFO

Excluded from the paper's original 29:

- Financial (7) — user constraint, no financial markets
- Sports (3) — LunarResearcher evidence (52% win rate ≈ random)

Agent roster

Name	Provider	Model ID
claude	anthropic	claude-opus-4-7
codex	openai	TBD — left as a placeholder constant in <code>config/agents.py</code> , finalized at build time

DB schema sketch (to be written into `db/schema.sql`)

Tables:

- **agents** — `id`, `provider`, `model_id`, `mode`, `starting_capital_cents`, `created_at`
- **markets** — `ticker` (PK), `category`, `title`, `settles_at`, `status`, `last_synced_at`
- **cycles** — `id`, `agent_id`, `started_at`, `ended_at`, `status`, `tool_call_count`, `prompt_tokens`, `completion_tokens`, `reasoning_tokens`, `wall_clock_seconds`
- **trades** — `id`, `agent_id`, `cycle_id`, `market_ticker`, `side`, `action`, `quantity`, `price_cents`, `fees_cents`, `executed_at`, `reasoning`, `realized_pnl_cents`
- **positions** — `id`, `agent_id`, `market_ticker`, `side`, `quantity`, `avg_entry_price_cents`, `opened_at`, `closed_at`, `close_reason`
- **settlements** — `id`, `agent_id`, `market_ticker`, `position_id`, `settled_at`, `realized_pnl_cents`, `winning_side`
- **snapshots** — `id`, `agent_id`, `cycle_id`, `taken_at`, `cash_cents`, `unrealized_pnl_cents`, `realized_pnl_cents_total`, `account_value_cents`, `open_position_count`, `win_rate_pct`, `max_drawdown_pct`

- **notes** — `id`, `agent_id`, `title`, `body`, `created_at`, `updated_at`
- **tool_calls** — `id`, `cycle_id`, `sequence_num`, `tool_name`, `arguments` (jsonb), `result` (jsonb), `duration_ms`, `called_at`

Amounts in **cents (integer)** to avoid float drift. Indexes on (`agent_id`, `market_ticker`), open positions, snapshots-by-time, and tool-calls-by-cycle.

Implementation order (when we leave plan mode)

0. **Revert** the partial scaffold (delete `toolshed/users/llewis/prediction_arena/`)
1. Re-scaffold the tree per this plan (README, requirements, `.env.example`, `.gitignore`, empty dirs, `__init__.py` files)
2. Config files (pure data, no logic)
3. DB schema + Supabase client
4. Exchange interface + **paper adapter first** (no Kalshi account needed to develop)
5. System prompt (separate deliverable, written next as a stand-alone task)
6. LLM base + Claude runner first (smaller surface than Codex)
7. Tools (trading, research, notes, `market_data`)
8. Orchestration cycle
9. `scripts/run_single_cycle.py` — manual end-to-end test
10. Scheduler — only after single-cycle works
11. Metrics + leaderboard
12. Kalshi live adapter (last — enables flipping to execute mode)

Frontend: out of scope. Defer until orchestration is working; inspect leaderboard via SQL/scripts in the interim.

Critical files (when implementing)

- `config/settings.py`, `config/markets.py`, `config/agents.py`, `config/cycle.py` — pure data, foundational
- `db/schema.sql` — concrete artifact, ready to apply
- `exchange/base.py` — interface contract that everything else depends on
- `prompts/system_prompt.md` — the verbatim prompt (next deliverable)
- `orchestration/cycle.py` — the heart of the system

Verification

After scaffolding:

1. `tree prediction_arena/` (or `ls -R`) matches the layout above.
2. `python -c "from prediction_arena.config import settings, markets, agents, cycle; print(markets.ALL_TICKERS)"` succeeds and prints 19 tickers.
3. `python -m pytest tests/ --collect-only` discovers test files (won't run since they're empty).
4. `python scripts/init_db.py` (once implemented) successfully applies `schema.sql` against the Supabase instance.

End-to-end verification happens at step 8 of the implementation order (`run_single_cycle.py`) — that's the first point the full stack is exercised together.

Resolved decisions

Question	Decision
Codex model identifier	Left vague — placeholder in <code>config/agents.py</code> , finalize at build time
Partial scaffold	Revert and re-scaffold cleanly (step 0 of implementation order)
DB target	Supabase (hosted Postgres)
Frontend	Deferred entirely — inspect leaderboard via SQL/scripts initially